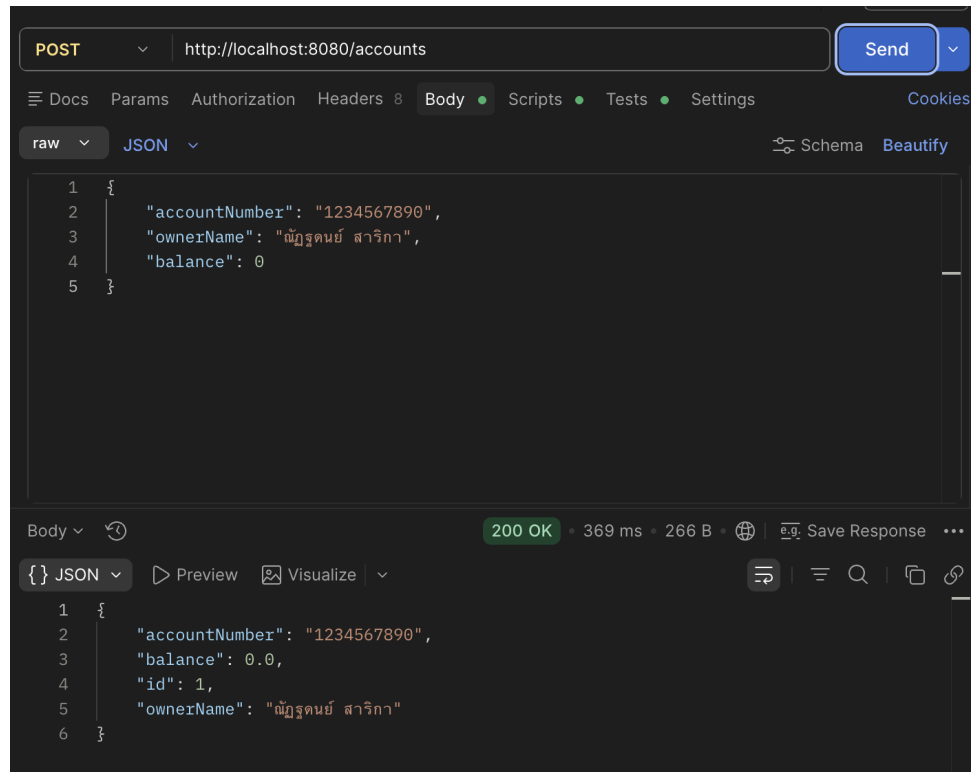


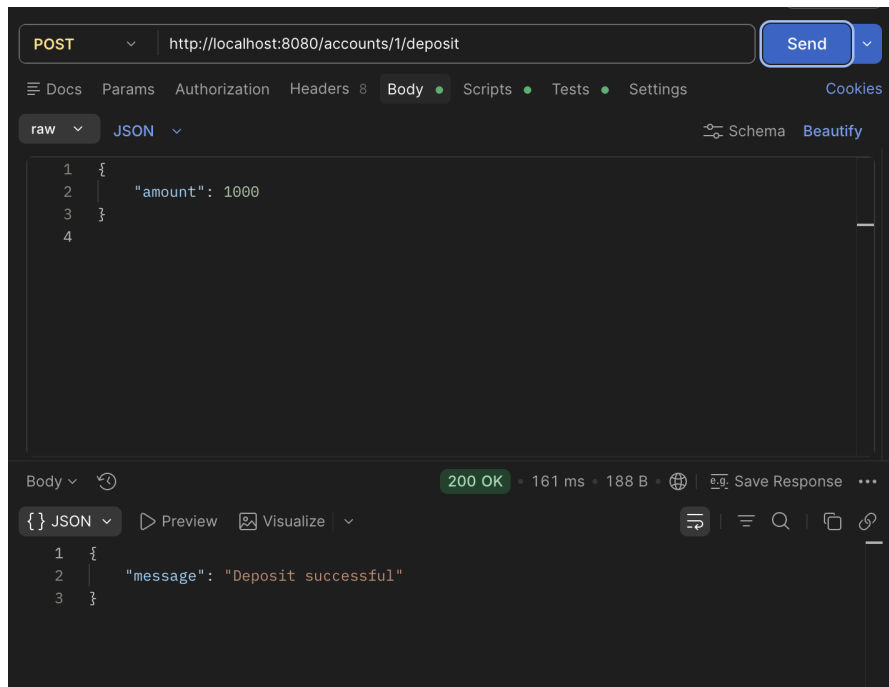
นาย ญัฐดนัย สารีกา 673380511-9 sec2

1.แคปหน้าจอ Response จาก Postman ที่แสดงว่า Account ถูกสร้างสำเร็จ และมี id ของ Account



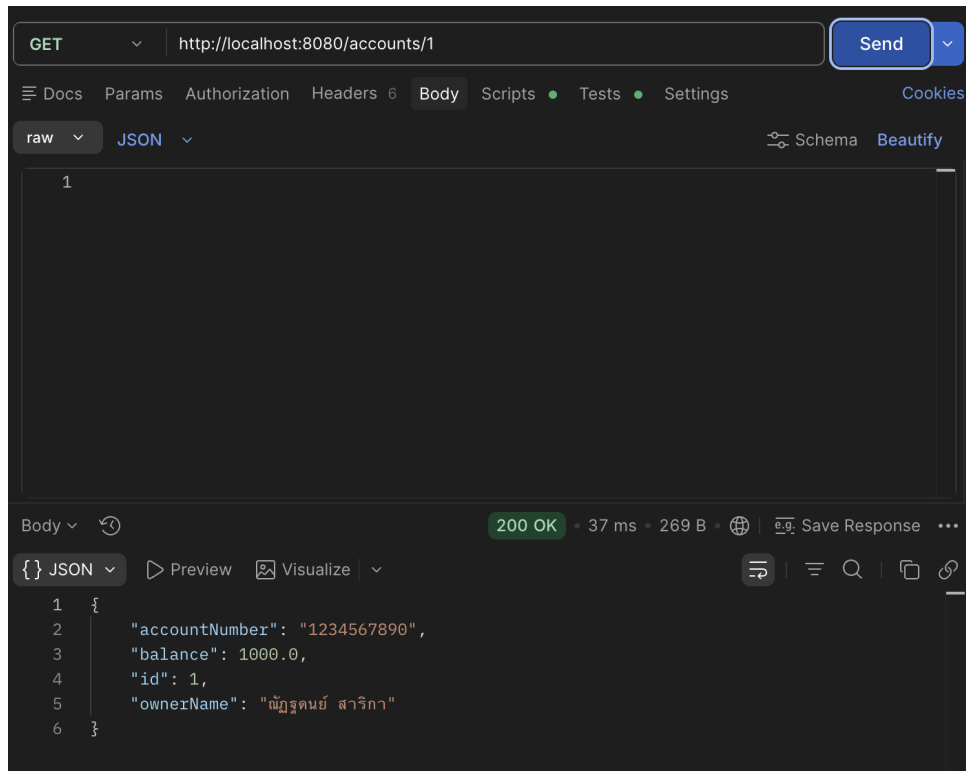
ทำการสร้าง account สำเร็จ

2.แคปหน้าจอ Request และ Response จาก Postman ที่แสดงว่าการฝากเงินสำเร็จ (ให้เห็นทั้งช่อง Body ที่ส่งไป และ Response ที่ได้กลับมา)



ทำการฝากเงินเข้าไปในบัญชี

3.แคปหน้าจอ Response ที่แสดงข้อมูล Account และ balance = 1000



ตรวจสอบจำนวนเงินในบัญชีพบว่าการฝากเงินในบัญชีสำเร็จ

4.แคปหน้าจอ ตาราง deposit_transaction ใน pgAdmin ที่แสดงรายการฝากเงิน

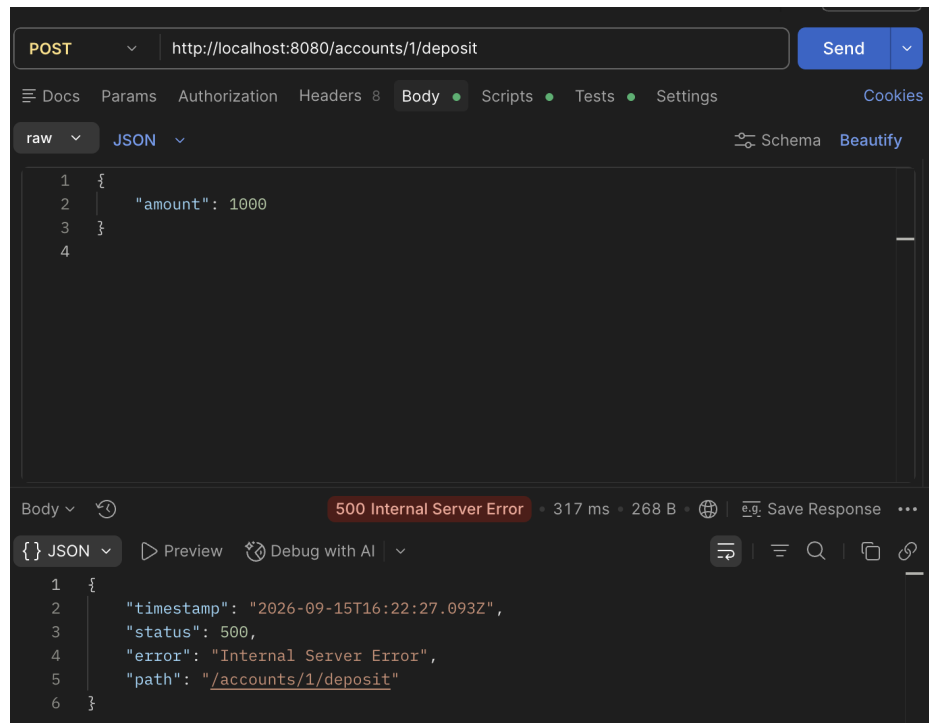
```
[postgres=# \q
[(base) draft@MacBook-Air-khxng-Draft ~ % psql -U postgres -d lab9
psql (16.14 (Homebrew))
Type "help" for help.

[lab9=# SELECT * FROM deposit_transaction;
 id | amount | account_id
----+-----+-----
  1 |  1000 |         1
(1 row)

lab9=#
```

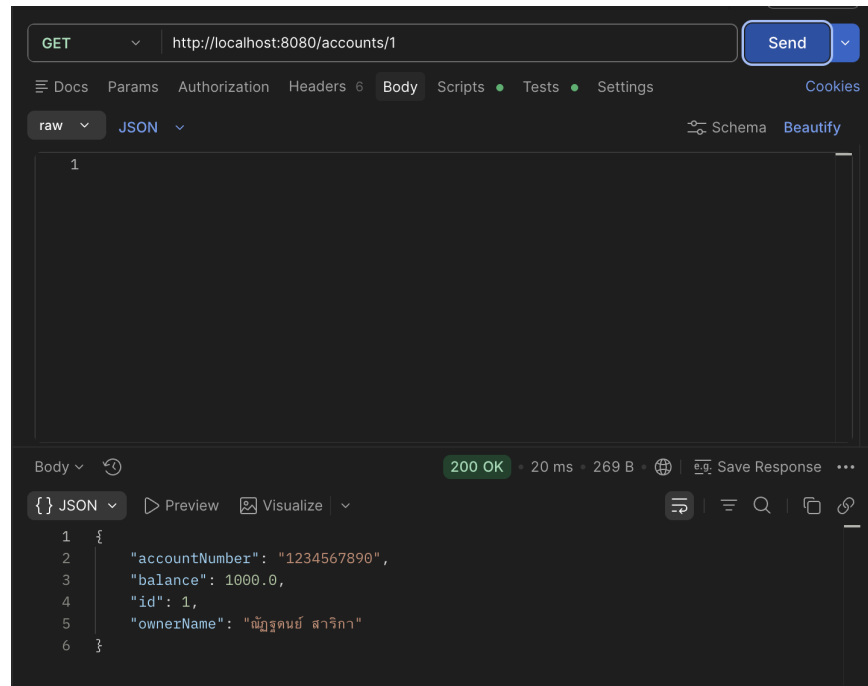
ทำการดูรายการข้อมูลใน Database

5. แคปหน้าจอ Postman ที่แสดง Status 500



ทำการโยน error เพื่อทดสอบระบบการ rollback โดยทำการเพิ่ม amount อีก 1000 เพื่อสังเกตผลว่าระบบจะบันทึกไหม

6. Response จาก Postman ที่แสดงว่า balance ไม่เพิ่ม



เห็นได้ว่าเมื่อเกิด error ระบบจะยกเลิกทุกธุรกรรมที่ทำอยู่ใน method นั้น

7.ตาราง deposit_transaction ใน pgAdmin ที่แสดงว่าไม่มีรายการใหม่

```
[lab9=# \q
[(base) draft@MacBook-Air-khxng-Draft ~ % psql -U postgres -d lab9
psql (16.14 (Homebrew))
Type "help" for help.

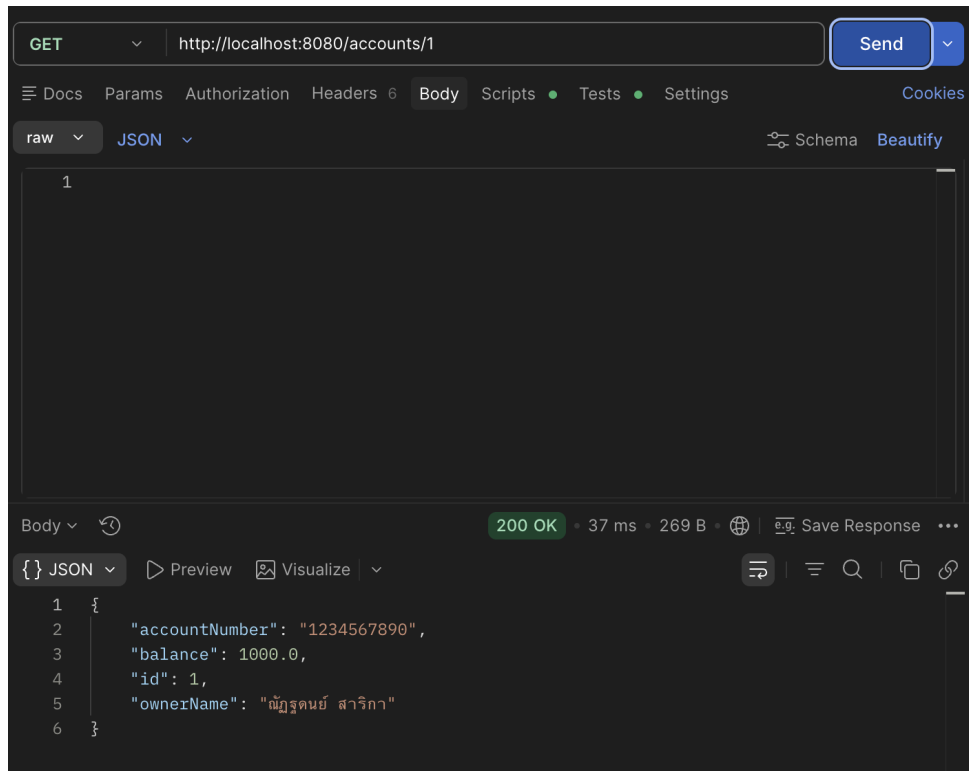
[lab9=# SELECT * FROM deposit_transaction;
 id | amount | account_id
-----+-----+-----
  1 |   1000 |           1
(1 row)

lab9=#
```

ทำการดูรายการข้อมูลใน Database หลัง rollback ไม่พบว่ามีการบันทึกข้อมูลลงไป

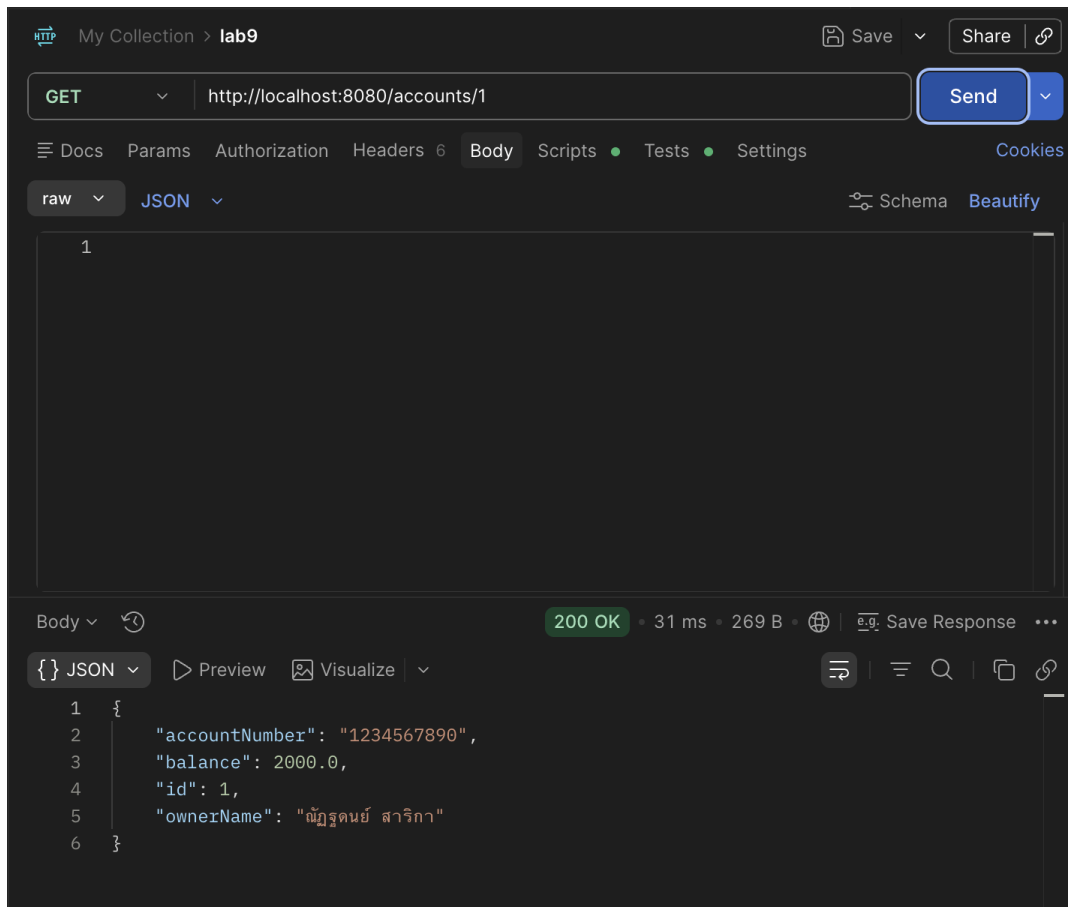
8.แคปผลการทดลองที่แสดงความแตกต่างระหว่าง

- ตอนที่รัน @Transactional



เห็นได้ว่าเมื่อเกิด error ระบบจะยกเลิกทุกธุรกรรมที่ทำอยู่ทำให้ amount เป็น 1000 เท่าเดิม

- ตอนที่ไม่มี @Transactional



เห็นได้ว่าเมื่อเกิด error ระบบจะไม่ยกเลิกธุรกรรมที่ทำอยู่ทำให้ amount เพิ่มขึ้นเป็น 2000

คำถามท้าย Lab

1.@Transactional มีหน้าที่อะไร?

ตอบ คอยคุมชุดคำสั่งที่ทำงานกับ Database ให้เป็นก้อนเดียวกันแบบ โดยจะบันทึกลงระบบต่อเมื่อทุกmethod ในฟังก์ชันสำเร็จ แต่ถ้าเกิด error จะทำการ rollback ทำให้ธุรกรรมทั้งหมดในก้อน transaction นั้นถูกย้อนกลับไปด้วย

2.เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

ตอบ ป้องกันข้อมูลผิดพลาด เช่น ยอดเงินถูกบวกเพิ่มไปแล้ว แต่ระบบล่มตอนกำลังบันทึกประวัติ

ฝาก หากใช้ Transaction ระบบจะยกเลิกการบวกเงินทันที ทำให้เงินไม่ลอย

3.COMMIT และ ROLLBACK ต่างกันอย่างไร?

ตอบ COMMIT: ยืนยันบันทึกข้อมูลลง Database อย่างถาวร เมื่อทุกขั้นตอนสำเร็จ

ROLLBACK: ยกเลิกการเปลี่ยนแปลงทั้งหมด แล้วย้อนข้อมูลกลับไปจุดเริ่มต้น เมื่อเกิด Error

4.Account และ DepositTransaction มีความสัมพันธ์แบบใด?

ตอบ One-to-Many (1:N): 1 บัญชี (Account) สามารถมีรายการฝากเงิน (DepositTransaction) ได้หลายรายการ

5.เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ตอบ ไม่มีการเปลี่ยนแปลงใดๆ เกิดขึ้น ระบบจะสั่ง ROLLBACK ดึงข้อมูลทั้งหมดกลับไปเป็นเหมือนตอนก่อนเริ่มรันโค้ดทันที

6.จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

ตอบ หากเกิด Error กลางทาง คำสั่งส่วนที่รันผ่านไปก่อนหน้านี้จะถูกบันทึกลง Database ไปแล้วโดยไม่ถูกยกเลิก ทำให้ข้อมูลค้างและขัดแย้งกัน